

Real-Time System Projects 2026

Karl-Erik Årzén

2026-02-24

1 Administrative Information

The projects should be performed as team works with the size of four persons per team. (In special cases it may be OK with smaller project teams.) Constraints on hardware, processes and supervisors may require synchronization among the projects. It may in certain cases be necessary for the project teams to reserve laboratory processes and/or computers using booking lists. More information about how this will be done in practice will be provided later.

The project teams should be organized and have handed in a priority list of their desired projects by **March 2**. The list should contain four projects in **priority order**. This should be done in the form of an email to **karlerik@control.lth.se** with the subject “Real-Time Project 2026”. The email should contain the **names** and the **email** addresses of **all** the members of the group. Please provide both the StiL addresses and the email addresses that you actually read. The email should also list all other control courses that the group members have taken. The projects will be assigned on March 3-4. We will contact you by email once this is done to plan the first meeting with your supervisor(s).

If you cannot find a group to join, then send us an email with your personal preferences and we will find or create a suitable group for you.

1.1 Suggested Solutions

A detailed suggested solution (3–4 pages) for the project should be presented to your supervisor no later than **March 27** (but preferably as soon as possible). You are not allowed to start working in the lab until your suggested solution has been approved. The suggested solution is also a natural basis for the project report.

The suggested solution could typically contain:

1. Introduction. What is the overall problem to be solved? What hardware (computers and processes), what operating system, and what programming language will be used in the implementation?

2. Program structure. What threads, communication and synchronization will be used? What are the main classes, and what are their public interfaces? Use some kind of graphical notation, e.g., UML or the graphical notation used in the Buttons exercise.
3. Operator communication. Screen layout, available commands in different modes, plotters, etc. Which parameters can be changed on-line by the user?
4. Control principles (if applicable). What controller structure will be used, how will the controller be designed, what reference generation will be used, etc.
5. Project plan. Time plan. Describe in which order the different parts will be implemented. Define what can be done in parallel. A Gantt chart is a good way to visualize the time plan.

However, the nature of the project may motivate a slightly different structure.

1.2 Project Requirements

Your project will be accepted if it passes the following requirements:

- A **program** that fulfills the specifications should have been demonstrated to your project supervisor by **May 18**. Each team member should be able to answer questions about the program structure, why a certain solution has been chosen, etc. All **code** must be commented, documented, and submitted to your supervisor (.zip file).
- A project **report** (in Swedish or English) should be written (using some word processing system). The suggested solution provides a good structure for the project report. The report should be handed in to your supervisor by Wednesday **May 18**. The supervisor will read the report and request revisions, if necessary. You are only allowed to hand in your report three times (including the first version). Spelling errors in the report are not accepted. The same holds for compression artefacts and black background in Simulink plots. Figures and plots should have clear labels and titles. The report normally contains the following parts:
 - A cover page with the name of the course, name of the project, the names and email addresses of the involved students, and the name of the supervisor.
 - An introduction that states the problem that has been solved.
 - A section describing the main program structure, both from a class and a real-time perspective. If possible illustrate this with some type of figure.
 - A section describing the control design aspects of the project.

- A section describing the user interface in the project including a short HowTo description on how to start and operate the program.
- A section containing the results. In case the project is a control-oriented project this should include plots of measurement signals, reference signal, and control signal. If the project is more of a real-time nature then this section could contain measurement results of different type.
- A conclusion section.

However, the nature of the project may motivate another outline.

- The project should be **demonstrated** during the project presentation lecture on **May 26**, 15:15-16:15.
- An **oral presentation** (10 minutes) should be made on the oral presentation session, **May 26**, 16:15–17:30. Prepare a few slides and show them on your own laptop or submit them to your supervisor at least one day before the presentation.

1.3 Important dates

- **March 2**: project groups and project priorities due.
- **March 3-4**: project assignments.
- **March 27**: suggested solution due.
- **May 18**: deadline for project report and demonstration to supervisors.
- **May 26**: demos and oral presentations.

1.4 Project Suggestions

A number of different projects are proposed below. The programming language used in the projects is Java, unless otherwise indicated. The character codes after the title have the following meaning:

- P** Pure real-time programming project. May not be selected by those who have taken the CS course EDAP10 Concurrent Programming.
- C** Programming in C, Python, or some other programming language required.
- 2** Small project. Only two persons.
- S** Special or research-related project. High risk factor. Only for highly motivated persons or PhD students.

Each project is described by a brief presentation. A more detailed specification of each project will be available when the projects start.

2 Control-Oriented Projects

Project 1 – Model Predictive Control Using CVXGEN or QPgen (S)

Implement a model predictive controller for a suitable lab process using CVXGEN or QPgen to generate fast code for embedded systems. Investigate the effects of prediction horizon on code size, execution speed and performance. You can also experiment with the use of constraints on the control signal and the output. Requires prior experience of MPC.

Project 2 – Autotuning of PID Controllers

The goal of the project is to implement relay-based automatic tuning and test it on some of our laboratory processes.

Project 3 – Advanced Control of the Ball-and-Beam Process (S)

Try MPC, LQG or adaptive control of the ball-and-beam process. With MPC or LQG you could try to move the ball as quickly as possible between the two end points of the beam. With adaptive control the goal could be to get the adaptive controller to converge before the ball falls off. If MPC is used prior experience is required.

Project 4 – Event-Based Sampling and PID Control

Most control theory is based on the assumption that equidistant sampling is used. This is typically not the way manual control is performed. There, a new control signal is only generated when necessary, e.g., when the control error has changed more than a certain limit or when the error derivative has changed more than a certain limit. Implement a PID controller that consists of two parts: one part that samples the input and performs limit checking with high frequency, and another part that, on demand, calculates a new control signal. Try the controller on some process in our lab, e.g., the DC servo or the double tank.

Project 5 – Vision Feedback

In this project you should use a camera as the sensor in a feedback loop. A suitable process can be the ball and beam process or the Furuta pendulum. For this project it is an advantage if you have taken a course in Image Analysis.

Project 6 – “Catch and Throw” Ball and Beam Process

An older version of the ball and beam process is equipped with a ball magazine and a solenoid that automatically pushes a ball onto the beam. The process

contains three different ball sizes. The task is to write a program that pushes a ball on to the beam, catches it, weighs the ball, and depending on the size of the ball either throws it into a waste basket or into a special ball basket mounted on the process. Handling of sequences and mode changes are important in the project.

Project 7 – “Catch and Throw” Ball and Beam Process in JGrafchart

The same as the previous project but you should use the Sequential Function Chart editor JGrafchart to implement the sequential parts of the control system, rather than doing it in Java. JGrafchart will communicate with a Java application where the feedback control loops are running using sockets.

Project 8 - Swingup and Control of a Furuta Pendulum

In this project you will develop a Java based control system for a rotating pendulum, also known as a Furuta pendulum. Your task is to develop an observer based state feedback controller that can operate in two different modes: control in the lower position and control in the upper (inverted) position. In either mode, it should be possible to specify a reference value for the arm angle. The control system should also be able to perform a swingup from the lower to the upper position using an algorithm that is provided to you. Relevant controller parameters and plots of the measurement and control signals should be accessible through a GUI developed by you. You will design and implement these controllers using the methods covered in the course and/or using linear quadratic methods (LQR/LQG). Knowledge of linear quadratic control (for example from the Advanced Course FRTN55) is a plus. You may not select this project if you already have worked with the Furuta pendulum, e.g., in the Project Course.

Project 9 - State feedback and time-optimal feedforward for the ball and beam process

In this project you should develop a state feedback + observer (with a disturbance observer for integral action) + time-optimal feedforward design for the ball and beam process that you used in Lab 1.

Project 10 – Stabilization of the Uni-directional MinSeg Segway (C)

The MinSeg is a small Segway-process that is built using, and programmed with, an Arduino. The uni-directional MinSeg has only one motor controller, i.e., it can only drive along a straight line. Your task is to implement a stabilizing controller for this process. The system should use wireless Bluetooth communication to a GUI that resides either on a desktop or on a smart phone where signals can be plotted and setpoints and controller parameters changed. This

project may not be selected by students who have taken the course FRTN55 Automatic Control, Advanced Course.

Project 11 – Control of the Crazyflie Quadcopter (CS)

Crazyflie (<http://www.bitcraze.io/crazyflie-2>) is a small open-source quadcopter. Your task is to develop a remote controller for the crazyflie. For this python can be used. It should allow the UAV to hover and to move between waypoints.

Project 12 – Crazyflie Control – Firmware Programming (CS)

Crazyflie (<http://www.bitcraze.io/crazyflie-2>) is a small open-source quadcopter. On the Crazyflie there is an Arm Cortex processor running FreeRTOS. This project proposes a re-implementation of parts of the firmware implementing the Crazyflie drone control system. Starting from the standard drone firmware (<https://github.com/bitcraze/crazyflie-firmware>), the project will explore alternative choices for the control algorithm as well as alternative choices for its real-time implementation. The aim is to evaluate the impact of different control design and real-time implementation choices.

Project 13– Control of a three-wheeled omni-directional robot (C)

The Omnibot is a three-legged mobile robot (a Delta-configuration). On each is attached a so called omni-wheel that allows movement in all directions. This allows very high positioning precision. In this project you should implement a controller for this robot using the on-board Raspberry processor.

Project 14 – Aperiodically Sampled Control

When controlling a process over a network, the combined latency caused by the computations and the communication can be very large and time-varying. Using an ordinary periodically sampled controller implementation might lead to a long sampling interval, so long that the performance of the control loop will be negatively affected, or even that the closed loop system becomes unstable. An alternative is then to sample as soon the control signal has been received from the network and actuated. This results in a sampling period that varies from invocation to invocation. In this project you should compare periodic and aperiodic control using some networked control loop.

Project 15 – MPC for the Furuta Pendulum (SC)

This project focuses on implementing a Model Predictive Control (MPC) system for stabilizing a Furuta pendulum in its upright position.

Project 16 – Analysing the effect of sensing delays on the Furuta pendulum (2-3C)

In this project, you will use state-space methods to control the desktop Furuta pendulum with the objective of balancing the pendulum in the inverted (upside-down) position. Additionally, you will introduce (time-varying) delays in the measurements - i.e. make the controller use 'old' data when regulating the system. The goal is to analyse how the system is affected and potentially adapt the controller to handle delays better. Knowledge of linear quadratic control (for example from the Advanced Course FRTN55) is a plus.

Project 17 – Control of the CloudGripper robot (C)

CloudGripper (cloudgripper.org) is a robotic manipulator located at KTH in Stockholm. The goal of this project is to control the robot remotely (over the cloud) and use vision feedback to make it perform a pick-and-place task. The programming language used in this project is Python.

3 Real-Time Programming Projects

Project 18 – Investigate Linux's scheduling policies (2PC)

Linux has several scheduling policies: `SCHED_OTHER`, `SCHED_FIFO`, `SCHED_RR`, and `SCHED_DEADLINE`. In this project you should do a deep dive into the Linux OS scheduler and explore the policies available. This includes investigating how to map the Java threads and thread priorities to native Linux processes and priorities. Then you should use this to implement some simple controller using, e.g., `SCHED_DEADLINE`.

4 Embedded Projects

In these projects you will implement your controller in an embedded computer, using a Linux PC for development and cross-compilation. On the PC you will also run the GUI for the controller, implemented in Java. The on-line communication between the host and the target machine should be based on RS-232 (serial cable) or Bluetooth.

Project 19 – Embedded Control of the Ball and Beam Process (C)

Design and implement a digital control system for the ball and beam process using the Atmel AVR processor. The controller should use state feedback from an observer and include integral action. Use fixed-point arithmetic for the control calculations.

5 AORTA projects

The department is currently part of a VINNOVA project called AORTA – Advanced Offloading of Real-Time Applications, together with Ericsson, Cognibotics, and Mälardalen University. The aim is to investigate different aspects of offloading real-time control applications to the cloud or to the edge. The focus of the project is robotics.

In the project a number of use-cases have been developed. The final use-case involves a mobile Sleipner robot that should move parts on a tray from one stationary robot to another stationary robot, avoiding obstacles on the floor. In order to do the path and trajectory planning a motion planning pipeline has been developed that can be offloaded to an edge server over a private 5G network in the department’s robot lab.

The Sleipner robot is an omnidirectional platform on which is mounted a two-armed Yumi robot from ABB. The first stationary robot that puts the parts on the tray is an HKM pick-and-place robot from Cognibotics and the stationary robot that removes the parts from the tray is another Yumi robot. Localization will be performed by a high-resolution camera mount on the ceiling looking down at the workspace and QR-like markers on the different robots, etc.

Within the use-case three smaller real-time systems projects have been defined. It is also possible to do larger and more realistic AORTA-inspired projects in the project course that runs in study period 4. In some cases the projects can be combined.

Project 20 – Yumi Learning to Grasp

The objective of this project is to design and implement a ROS-based control program for the ABB YuMi robot. The robot must autonomously grasp a tray from a table, transport it in a stable manner despite external disturbances, and place it at a specified target location upon receiving a command.

The system should perform the following tasks:

- Detect and grasp the tray from the table, using a force sensor
- Maintain a stable grasp while carrying the tray, even in the presence of disturbances.
- Place the tray at a desired location when instructed.

The following input information is provided to your program:

- The center position of the tray and its geometric dimensions.
- The current joint angles of the YuMi robot.
- A control signal indicating whether the robot should carry the tray or place it down.

Your program must generate the desired joint velocities for the YuMi robot.

All communication and control must be implemented using the Robot Operating System (ROS).

Project 21 – Camera Vision

The objective of this project is to design and implement a computer vision algorithm that detects and classifies objects in the workspace using a camera and visual markers (ArUco markers). The system must identify the robot, a tray, and obstacles, and estimate their positions and geometric dimensions.

The system should perform the following tasks:

- Detect and classify objects as one of the following:
 - Robot
 - Tray
 - Obstacles
- Estimate the position and geometric dimensions of each detected object.
- Represent non-convex obstacles as a union of convex shapes.

The following input information is provided to your program:

- A video stream from a camera using <https://gitlab.control.lth.se/processes/omnibot/omnibot-webcam.py>.

Your program must generate:

- The position and dimensions of the robot.
- The position and dimensions of the tray.
- A list of convex obstacles, including their positions and dimensions.

All obstacles must be represented using one or more of the following convex shapes: circles, oriented boxes, triangles.

It is recommended to implement the solution in Python using OpenCV and ArUco markers.

Project 22 – Omnibot On the Move

In this project, your task is to implement a control algorithm that enables a mobile robot to follow a path generated by a higher-level motion planner, utilizing feedback from the lighthouse positioning system. Here we will use an Omnibot roble instead of Sleipner. The control algorithm should incorporate a feedforward structure that uses both position and velocity trajectories.

The Omnibot is a robot developed here at the Department of Automatic Control in Lund. You will control the robot and request information from it using the Omnibot client (see <https://gitlab.control.lth.se/processes/omnibot/omnibot.py>).

An existing motion planner accepts an initial position (x, y, θ) , a target end position, and a list of obstacles. It returns trajectories for position, velocity, and acceleration over time. These trajectories are calculated to avoid obstacles and aim to provide the shortest path or motion between the start and end positions.

6 Your Own Ideas

If you feel that you have an idea for a suitable project, it can always be discussed.